

Analyse détaillée de la flotte — supervision

Généré le 2026-07-24 19:02 — étage déterministe (scan 0 token) + étage qualitatif (audit-technique, lecture du code).

VSCode — Bac à sable proto PPT (COMOP, Node.js)

Dernier commit : 2026-07-24T17:46:20+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	ok	5 fichier(s) de test, coverage configuré
Test fonct.	ok	2 test(s) à vérification réelle
Revue code	ok	hook pré-commit, bmad-code-review
Revue incr.	ok	skill + hook SessionStart
Design	moyen	deck-design-library, ppt-designer
Doc	moyen	wiki, CLAUDE.md
Cadrage produit	moyen	besoins + brief BMAD
Pratiques+rules	moyen	CLAUDE.md
Sécu (proxy)	ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ok

Commentaire : Gestion d'erreurs globalement solide (validation Mandatory + Test-Path côté PowerShell, garde-fous de taille et de chemin côté Node), avec quelques incohérences mineures de codes d'erreur.

EXEMPLE CONSTATÉ	LOCALISATION
JSON malformé -> 500 générique au lieu de 400 : JSON.parse non entouré d'un try/catch local	<code>server.js:303,265</code>

Incohérence : safeTemplatePath try/catché en 400 ailleurs mais pas dans /api/generate (template invalide -> 500)

server.js:304

Générateur PS bien gardé : paramètres Mandatory, Test-Path sur TemplatePath/DataPath, nettoyage workDir en finally

src/generate-comop.ps1:1-12,34-39,86-89

process.exit(1) sur uncaughtException : un crash isolé tue tout le serveur (pas de dégradation gracieuse)

server.js:376-384

performance — ok

Commentaire : Aucun enjeu réel pour un prototype local : un process PowerShell par requête et zip extrait/recomprimé à chaque appel, acceptable à cette volumétrie.

EXEMPLE CONSTATÉ

LOCALISATION

Spawn d'un powershell.exe complet + extraction/recompression zip du template à chaque génération : coûteux mais non critique à cette échelle

server.js:88-106,317-326

risque_technique — moyen

Commentaire : Proto assumé mais fragilisé par l'absence de package.json malgré Node, l'absence de CI et de tests unitaires (un seul smoke-test), et de la duplication de fonctions entre scripts PowerShell.

EXEMPLE CONSTATÉ

LOCALISATION

Aucun package.json malgré un serveur Node : pas de manifeste ni version de moteur déclarée (stdlib uniquement, zéro dépendance)

comop-pptx-prototype/ (racine)

Couverture limitée à un seul smoke-test.ps1 (14 assertions e2e), aucun test unitaire ni CI

src/smoke-test.ps1

Duplication de helpers (Get-ShapeName, Get-GraphicType, New-TempDirectory) copiés entre scripts

src/detect-template-zones.ps1:10-51
vs remove-template-shape.ps1:14-27

Manipulation du PPTX par regex sur le XML plutôt qu'un parseur : fragile face aux variations OOXML

src/generate-comop.ps1:61-66

remove-template-shape écrase le template source (opération destructive sans sauvegarde)

src/remove-template-shape.ps1:74-77

securite — moyen

Commentaire : Le risque n°1 (injection de commande Node→PowerShell) est bien traité (spawn en tableau d'arguments, chemins assainis, XML échappé), mais le serveur écoute sur toutes les interfaces (0.0.0.0) alors qu'il exécute PowerShell et accepte des templates uploadés — exposition LAN non voulue.

EXEMPLE CONSTATÉ	LOCALISATION
Serveur exposé sur toutes les interfaces : <code>server.listen(port)</code> sans hôte -> écoute 0.0.0.0 ; le service qui exécute PowerShell et accepte l'upload de templates est accessible depuis le LAN (le README parle de localhost)	<code>server.js:386</code>
Invocation sûre : <code>spawn('powershell.exe', [args])</code> en tableau, sans <code>shell:true</code> ni interpolation -> pas d'injection de commande	<code>server.js:88-106</code>
Path traversal gardé : <code>safeTemplatePath</code> impose <code>basename+.pptx+startsWith(templatesDir)</code> ; <code>serveStatic</code> vérifie <code>startsWith(webDir)</code>	<code>server.js:108-115, 354-355</code>
Données utilisateur échappées (<code>SecurityElement::Escape</code>) avant insertion XML -> pas d'injection OOXML	<code>src/generate-comop.ps1:16-26</code>
ExecutionPolicy Bypass systématique + extraction de zip fourni par l'utilisateur : surface résiduelle (zip-slip théorique selon version .NET)	<code>server.js:90 ; src/generate-comop.ps1:47</code>

VSCode1 — Questionnaire maturité agile/produit + export PPT

Dernier commit : 2026-07-24T19:00:37+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	ok	12 fichier(s) de test, coverage configuré
Test fonct.	moyen	1 test(s) à vérification réelle
Revue code	ok	agent reviewer, hook pré-commit, bmad-code-review
Revue incr.	ok	skill + hook SessionStart
Design	ok	deck-design-review, deck-design-library, ppt-designer
Doc	ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	moyen	persona, why
Pratiques+rules	ok	linter, CI, CLAUDE.md, conventions
Sécu (proxy)	ok	deny rules, guard git

Audit qualitatif du 2026-07-24 (lecture du code réel)

robustesse — ok

Commentaire : Validation des entrees systematique et re-import referentiel transactionnel/idempotent bien maitrise, a quelques atomicites et echecs silencieux mineurs pres (inchange).

EXEMPLE CONSTATÉ	LOCALISATION
replacelnvites fait DELETE puis INSERT en boucle hors transaction : un echec en cours laisse les invites partiellement remplaces (non atomique, contrairement a reconcileReferentiel)	app/src/invites.js:59-65
Nettoyage des fichiers temp d'export silencieux : unlink().catch(()=>{}) ignore l'echec, fuite possible dans os.tmpdir()	app/src/server.js:868-871
sessionStatus compare des getTime() non gardes : une date stockee invalide donnerait NaN et un statut 'ouverte' par default sans erreur	app/src/server.js:42-49

performance — ok

Commentaire : Le N+1 d'agregerResultats est RESOLU (une requete IN groupee par question + Map repondants au lieu de find() lineaire) — l'export departement en profite mecaniquement (~3x(N+1) -> quelques requetes). Restes marginaux assumes : double appel agregerResultats a l'export departement (desormais bon marche) et absence de pagination (volumetrie faible).

EXEMPLE CONSTATÉ	LOCALISATION
RESOLU (2026-07-24) : N+1 remplace par une requete 'WHERE repondant_id IN (...)' groupee par question ; lookup repondant en Map — verifie par npm test complet vert	app/src/server.js:agregerResultats
Reste assume : agregerResultats appele 2x a l'export departement (redondance désormais bon marche : ~2 requetes par appel)	app/src/server.js:708-803
Reste assume : aucune pagination sur les listings — impact limite par la volumetrie	app/src/server.js:109-114,166-173

risque_technique — moyen

Commentaire : Les 2 findings de test sont RESOLUS (coeur de calcul teste via scores.js/test-scores.js ; livrable PPT branche dans npm test via un pont qui skip proprement en CI). Restent : duplication des routes /equipes/departements et couplage au singleton db.

EXEMPLE CONSTATÉ	LOCALISATION

RESOLU (P2, 2026-07-24) :
moyenneDe/statsNiveaux(ecart-type
population)/deltaHistorique extraits en
fonctions pures dans src/scores.js et testes par
scripts/test-scores.js (cas limites : null ignores,
ecart-type [0,4]->2, sqrt(2/3), delta null/zero) ;
branche dans npm test

app/src/scores.js + app/scripts/test-scores.js

RESOLU (2026-07-24) : test-export-ppt.py
branche dans npm test via scripts/test-export-
ppt-si-dispo.js (execute si python-pptx
importable, SKIP explicite sinon — CI ubuntu
verte, poste outil teste le PPT reel)

app/package.json: 'test' + app/scripts/test-export-
ppt-si-dispo.js

Duplication quasi identique des routes
/equipes et /departements et du motif
manager=== 'sans' repete ~6 fois

app/src/server.js:432-466,532,592,622

Couplage fort au singleton db importe
directement par tous les modules —
testabilite reduite

app/src/referentiel.js:2,invites.js:2,server.js:9

securite — moyen

*Commentaire : SQL entierement parametre, pas de shell ni d'eval et aucun secret en clair, mais l'API —
qui expose des donnees nominatives (nom/prenom/email) — n'a strictement aucune authentication
(Epic 10 non implemente, decision produit). Inchange.*

EXEMPLE CONSTATÉ	LOCALISATION
AUCUNE authentication/autorisation sur toute l'API : n'importe qui atteignant le serveur peut lister les repondants avec PII, creer/supprimer/fusionner, exporter	app/src/server.js (toutes les routes /api/*)
SQL sain : requetes parametrees partout ; interpolations = liste blanche CHAMPS_FUSIONNABLES + placeholders generes — pas d'injection	app/src/server.js:164-186,776 ; referentiel.js:276-286
.env.dev/.preprod/.prod committes alors que .gitignore ne les ignore pas (ici sans secret : ports/chemins — mais mauvaise pratique)	.gitignore ; app/.env.dev, .preprod, .prod
Export PPT via execFile(python,[script,...]) SANS shell (pas d'injection)	app/src/server.js:867,880

VSCode2 — Interview-to-Deck (FastAPI)

Dernier commit : 2026-07-24T19:00:03+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	ok	31 fichier(s) de test, coverage configuré
Test fonct.	ok	19 test(s) à vérification réelle
Revue code	ok	hook pré-commit, bmad-code-review
Revue incr.	ok	skill + hook SessionStart
Design	ok	deck-design-review, deck-design-library
Doc	ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	moyen	persona, besoins
Pratiques+rules	ok	linter, CI, CLAUDE.md, conventions
Sécu (proxy)	ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-24 (lecture du code réel)

robustesse — ok

Commentaire : Gestion d'erreur solide et homogene (hierarchie AIError/TranscriptionError, timeout+retry cible, parsing JSON tolerant), le seul vrai trou etant l'absence de validation des uploads audio (inchange).

EXEMPLE CONSTATÉ	LOCALISATION
Upload audio sans validation de taille ni de type MIME : seul l'import .docx verifie l'extension — DoS memoire possible	app/routers/interviews.py:935,1382,951
Les 'except: pass' comptes par ruff (S110) sont les warm-ups best-effort documentes du lifespan, pas des echecs silencieux reels	app/main.py:54-59
Bon point : garde-fou anti-path-traversal explicite au telechargement, parsing JSON IA tolerant	app/routers/interviews.py:979-980 ; app/services/ai_common.py:278-293

performance — ● ok

Commentaire : Les 2 findings majeurs sont RESOLUS : le travail CPU-bound (Whisper, extraction IA) passe par `asyncio.to_thread` (la boucle d'evenements ne gele plus pendant des minutes) et l'audio complet est streamé par blocs vers le disque au lieu de passer en RAM. Reste une opportunité mineure (map-reduce IA séquentiel) et le bon point historique (transcription parallèle `ProcessPoolExecutor`).

EXEMPLE CONSTATÉ	LOCALISATION
RESOLU (2026-07-24) : transcribe_segment/transcribe_notes/import_interview basculent le travail CPU/IA sur <code>asyncio.to_thread</code> — vérifie par la suite complète (330 passed)	<code>app/routers/interviews.py</code>
RESOLU (2026-07-24) : <code>save_record_backup</code> streamé l'upload par blocs de 1 Mio (<code>shutil.copyfileobj</code> dans un thread) au lieu de charger 1h30-3h d'audio en RAM	<code>app/routers/interviews.py:save_record_backup</code>
Reste (opportunité, pas un risque) : appels IA en boucle séquentielle sur les tronçons (map-reduce non parallélise)	<code>app/services/interview_libre_extract_ai.py:215-223</code>
Bon point : transcription découpée et parallélisée sur <code>ProcessPoolExecutor</code> (~1,8x), jobs en <code>BackgroundTasks</code>	<code>app/services/audio_transcribe.py:161-179</code>

risque_technique — ● moyen

Commentaire : `deps` épinglées (RESOLU), `budget_ok` factorisée (RESOLU), B904 corrigées (RESOLU). Restent : gros module `pptx_export` (1636 l) et migrations `SQLite` en `f-string` — dette assumée, documentée.

EXEMPLE CONSTATÉ	LOCALISATION
RESOLU (2026-07-24) : les 12 dépendances passent de <code>>= a ==</code> aux versions du <code>venv</code> (<code>pip freeze</code>) ; les 2 <code>>=</code> restants sont dans des commentaires. build reproductible (<code>pip check</code> OK, <code>dry-run</code> résolvant, 330 passed après épinglage)	<code>requirements.txt</code> ; <code>requirements-dev.txt</code>
RESOLU (2026-07-24) : <code>budget_ok</code> factorisée en fabrique unique <code>_budget_lignes</code> (2 des 3 définitions ; la 3e garde sa logique propre documentée) + B904 x2 corrigées (from exc)	<code>app/services/pptx_export.py:_budget_lignes</code> ; <code>app/routers/synthese.py</code>
<code>pptx_export.py</code> = 1636 lignes (gros module d'export) ; importe bien <code>pptx_deck</code> as <code>D</code> (pas de duplication massive) — dette de taille assumée	<code>app/services/pptx_export.py</code>

Migrations SQLite additives en f-string DDL :
constantes internes (pas d'injection) mais
mecanisme fragile (pas d'outil, pas de rollback)

app/db.py:53-85

securite — ● ok

Commentaire : Aucune cle en dur, .env gitigne, aucun vecteur d'injection dangereux (pas de shell=True/eval/pickle/yaml.load/os.system), ORM parametre et fichiers uploades renommes cote serveur (inchange).

EXEMPLE CONSTATÉ	LOCALISATION
Cles API strictement en variables d'environnement, .env.example en placeholders, .env gitigne	app/services/ai_common.py:35-43 ; .gitignore:8
Aucun appel dangereux : grep shell=True/eval/pickle/yaml.load/os.system => 0 resultat dans app/ et tools/	app/ (grep global)
Fichiers uploades ecrits sous un nom genere serveur (mission_id+timestamp+uuid), jamais le filename client ; telechargement protege du path-traversal	app/routers/interviews.py:963- 964,979-980
Point mineur : absence de plafond de taille d'upload => risque theorique DoS memoire plutot que faille d'integrite	app/routers/interviews.py:941,956

VSCode3 — Cadrage BMAD IAP (deck de synthèse)

Dernier commit : 2026-07-24T18:15:00+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	● moyen	3 fichier(s) de test, pas de coverage
Test fonct.	● ok	2 test(s) à vérification réelle
Revue code	● ok	hook pré-commit, bmad-code-review
Revue incr.	● ok	skill + hook SessionStart
Design	● ok	deck-design-review, deck-design-library, ppt-designer

Doc	● moyen	wiki+html, CLAUDE.md
Cadrage produit	● moyen	why
Pratiques+rules	● moyen	CLAUDE.md, conventions
Sécu (proxy)	● ok	deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ● moyen

Commentaire : La régression NameError du build() est bien réparée et l'échec réseau des photos a un repli propre, mais le template est chargé sans garde dès l'import, seul vrai point de fragilité restant.

EXEMPLE CONSTATÉ	LOCALISATION
Template chargé sans garde au niveau module (import) et dans new_prs() — un template-octo.pptx absent lève un FileNotFoundError brut ; importer generate_deck (ce que fait le test) suffit à casser	docs/cadrage- ppt/generate_deck.py:96,112
Régression historique corrigée : le build() v2.6 appelle uniquement des slide_* définies (plus de NameError du commit 26461d9 mentionné dans CLAUDE.md)	docs/cadrage- ppt/generate_deck.py:2624-2758
Bon réflexe anti-échec-silencieux sur les photos (try/except + repli nature_images + print) ; le repli du repli n'est pas gardé	docs/cadrage- ppt/generate_deck.py:287-298
Contenu 100% codé en dur (pas de source externe) : « données manquantes » ne s'applique pas ; géométrie vérifiée et remontée (exit 1)	docs/cadrage- ppt/generate_deck.py:2752-2758

performance — ● ok

Commentaire : Pas d'enjeu perf significatif (génération de deck statique) — seul coût réseau (photos Openverse) mis en cache disque.

EXEMPLE CONSTATÉ	LOCALISATION
Fetch réseau des photos mis en cache sur disque (génére une seule fois si _img/<scene>.png existe)	docs/cadrage- ppt/generate_deck.py:283-284

risque_technique — ● moyen

Commentaire : Générateur monolithique de 2767 lignes reposant sur des copies manuelles d'utilitaires

de projets frères (duplication à resynchroniser à la main), mais le chemin de génération est réellement couvert par un test fonctionnel qui build + rend.

EXEMPLE CONSTATÉ	LOCALISATION
generate_deck.py = monolithe de 2767 lignes, ~35 fonctions slide_* dans un seul fichier, sans découpage par chapitre	docs/cadrage-ppt/generate_deck.py (2767 l)
Duplication inter-projets par copie : pptx_deck.py copie du skill global, framed_image/nature_images/stock_images greffés via sys.path — resynchro manuelle, dérive possible	docs/cadrage-ppt/pptx_deck.py ; generate_deck.py:66-70
Bonne couverture du chemin de génération : test_generate_deck.py build() + 40 slides + géométrie + cadres photo + rendu réel LibreOffice→PDF	docs/cadrage-ppt/test_generate_deck.py:90-193
Code mort assumé : slide_sous_chapitre conservé sans appelant depuis v2.6 (documenté)	docs/cadrage-ppt/generate_deck.py:164

securite —  ok

Commentaire : Surface faible confirmée par grep : aucun shell=True/eval/pickle, subprocess uniquement en arguments-liste, aucun secret réel en dur, chemins dérivés de __file__ et de littéraux internes.

EXEMPLE CONSTATÉ	LOCALISATION
Aucun shell=True, eval(), ni pickle dans le périmètre — vérifié par grep	grep sur docs/cadrage-ppt, tests, scripts
subprocess uniquement en listes d'arguments (rendu LibreOffice du test, inventaire git) — pas d'injection	docs/cadrage-ppt/test_generate_deck.py:70-73
Aucun secret en dur : les occurrences « Secrets »/« credential » sont du contenu de slide (classification D4), pas des valeurs	docs/cadrage-ppt/generate_deck.py:658
Chemins non assainis mais non exposés : noms d'images dérivés de littéraux internes fixes et de __file__, pas d'entrée utilisateur	docs/cadrage-ppt/generate_deck.py:283

VSCode4 — Deck OHC RH dispositifs d'écoute (pré-code)

Dernier commit : 2026-07-24T10:05:49+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	● moyen	1 fichier(s) de test, pas de coverage
Test fonct.	● moyen	1 test(s) à vérification réelle
Revue code	● ok	hook pré-commit, bmad-code-review
Revue incr.	● ok	skill + hook SessionStart
Design	● ok	deck-design-review, deck-design-library, ppt-designer
Doc	● moyen	wiki+html, CLAUDE.md
Cadrage produit	● absent	aucun artefact de cadrage produit détecté
Pratiques+rules	● moyen	CLAUDE.md
Sécu (proxy)	● ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ● moyen

Commentaire : Bon self-check bloquant et repli hors-ligne pour les photos Openverse, mais crashes durs non gardés sur template/média absents et quelques échecs « print-and-continue » silencieux.

EXEMPLE CONSTATÉ	LOCALISATION
Accès aux blobs média par clé sans garde : si la v6 (zip) n'a pas exactement les 5 image*.png attendues, KeyError brut et non explicite	<code>generate_deck_ohc.py:513-515, 528 ;</code> <code>charger_media:85-92</code>
Aucune garde sur l'absence du template v6 : Presentation(SOURCE_V6)/charger_media lèvent FileNotFoundError non capturée si le fichier source manque	<code>generate_deck_ohc.py:1109-1110 ;</code> <code>SOURCE_V6:63-65</code>
Fetch Openverse correctement replié : try/except -> nature_images hors-ligne, timeouts 15s/20s côté stock_images — point fort	<code>generate_deck_ohc.py:271-279 ;</code> <code>stock_images.py:40, 52</code>
Cadres photo vides gérés défensivement, mais l'image manquante ne fait PAS échouer build() — semi-silencieux (seul le test le rattrape)	<code>generate_deck_ohc.py:262-264 ;</code> <code>_repositionner_layout_chapitre:345-347</code>

Self-check géométrie + débordements texte BLOQUANT
en fin de build (exit 1) — filet solide

`generate_deck_ohc.py:1143-1146 ;`
`pptx_deck.verifier_geometrie:723`

performance — ● ok

Commentaire : I/O réseau et disque bien maîtrisés : photos Openverse mises en cache sur disque (fetch une seule fois), zip et template ouverts une seule fois par build.

EXEMPLE CONSTATÉ	LOCALISATION
Fetch réseau caché sur disque : <code>os.path.exists</code> court-circuite tout re-téléchargement — 4 requêtes à froid, 0 ensuite	<code>generate_deck_ohc.py:271-274</code>
Extraction zip de la v6 faite UNE fois (<code>charger_media</code> appelé une seule fois), pas par slide	<code>generate_deck_ohc.py:1109 ;</code> <code>charger_media:85-92</code>
4 fetch réseau séquentiels à froid — non parallélisés mais bornés et cachés, coût négligeable	<code>generate_deck_ohc.py:1119,1126,1132,1137</code>

risque_technique — ● moyen

Commentaire : `pptx_deck.py` est une copie explicite du module `VSCoDe2` (duplication cross-projet), avec un générateur de 1161 lignes truffé de magic numbers de layout et de noms de formes template fragiles, couvert par un seul fichier de test d'intégration.

EXEMPLE CONSTATÉ	LOCALISATION
Duplication assumée : <code>pptx_deck.py</code> (930 l) est une copie de <code>VSCoDe2</code> — risque de divergence entre les 3 copies, aucune source unique	<code>pptx_deck.py:7-8 (docstring)</code>
Couplage fragile aux noms exacts de formes/layouts du template ('Google Shape;201;p40', '51 - Chapitre [2]') : un changement de template casse en silence	<code>generate_deck_ohc.py:322-331,341-349 ; _layout:95-99</code>
Magic numbers de layout omniprésents ; ~8 cartes KPI dupliquées quasi à l'identique (fonctions slides très longues)	<code>generate_deck_ohc.py:519-622 (slide_personas ~100 l)</code>
Imports cross-skill via <code>sys.path.insert</code> (<code>framed_image/nature_images/stock_images</code> depuis <code>.claude/skills</code>) : couplage chemin-relatif au REPO	<code>generate_deck_ohc.py:54-58</code>
Couverture : ~40 assertions d'un seul fichier d'intégration pour ~2100 lignes ; état global mutable <code>_IDS_BLOB = iter(...)</code> (StopIteration possible)	<code>test_generate_deck_ohc.py (unique) ;</code> <code>generate_deck_ohc.py:148</code>

securite — ● ok

Commentaire : Aucun eval/pickle, subprocess LibreOffice en forme-liste sans shell=True, fetch réseau avec timeouts et slug assaini ; seul bémol, aucune allowlist de schéma ni plafond de taille sur l'URL image renvoyée par l'API.

EXEMPLE CONSTATÉ	LOCALISATION
subprocess (LibreOffice, test uniquement) en forme-liste avec timeout=120, SANS shell=True — pas d'injection	test_generate_deck_ohc.py:67-71
Aucun eval / pickle dans tout le périmètre scripts (grep vide)	scripts/
Téléchargement de l'URL image de l'API Openverse via urllib.urlopen sans allowlist de schéma ni plafond sur r.read() ; atténué par endpoint HTTPS de confiance et timeout 20s	stock_images.py:50-53
Écritures fichier bornées à des chemins fixes ; la requête utilisateur est assainie en [alnum/_] avant usage comme nom de fichier — pas de traversée	generate_deck_ohc.py:66-68,241-242

Vscode5 — Supervision multi-projets (ce projet)

Dernier commit : 2026-07-24T18:36:34+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	● moyen	6 fichier(s) de test, pas de coverage
Test fonct.	● ok	2 test(s) à vérification réelle
Revue code	● ok	hook pré-commit, bmad-code-review
Revue incr.	● absent	absente
Design	● n/a	ne produit pas de deck
Doc	● ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	● ok	persona, why, besoins, valeur + brief BMAD
Pratiques+rules	● moyen	linter, CLAUDE.md

Audit qualitatif du 2026-07-24 (lecture du code réel)

robustesse — ● moyen

Commentaire : Le serveur est défensif là où il compte (fail-open des jobs, garde-fou anti-doublon, isolation par env dans les scripts) mais un en-tête Content-Length malformé casse la requête, et les échecs de sous-process restent muets au-delà du statut.

EXEMPLE CONSTATÉ	LOCALISATION
Content-Length non numérique (ex. 'abc') leve ValueError HORS du try/except -> 500 au lieu d'un 400 propre	scripts/serve_wiki.py:312
_run_job capture toute Exception en 'erreur (...)' sans distinguer binaire introuvable / timeout / crash du sous-process — diagnostic pauvre	scripts/serve_wiki.py:247
JOBS croît sans bornes en mémoire (jamais purge) — négligeable en usage local mais une session très longue accumule tous les jobs	scripts/serve_wiki.py:214

performance — ● ok

Commentaire : Aucun enjeu : serveur local mono-utilisateur, jobs en threads, sortie bornée à 80 lignes ; le rescan post-validation est --no-refresh (0 relance des scans lourds).

risque technique — ● moyen

*Commentaire : Le duo scan_projets.py (~2000 l) + serve_wiki.py concentre beaucoup de JS inline généré depuis Python — deux bugs d'échappement Python->JS déjà vécus ce jour (\' consomme par Python, * / \s en SyntaxWarning) ; ce JS n'est testable que par harnais Edge externe, jamais en CI.*

EXEMPLE CONSTATÉ	LOCALISATION
Le JS du wiki est une grosse chaîne générée dans render_html : deux bugs d'échappement Python->JS en une journée (dont un cassant la syntaxe JS entière) — fragile, pas de garde à la génération	scripts/scan_projets.py:render_html (bloc <script>)
serve_wiki.py sans test permanent jusqu'à ce cycle ; tests/test_serve_wiki.py (ajoute aujourd'hui) couvre le HTTP mais PAS le JS généré	tests/ (pas de test du <script> genre)

sécurité — ● ok

Commentaire : Fondamentaux solides (bind 127.0.0.1, argv en liste jamais shell=True, allowlist d'actions, cibles injectées en prompt pas en shell) ; les 3 trous relevés par cet audit ont été CORRIGÉS dans la

foulee (echappement XSS, CORS restreint localhost/null, corps POST borne + Content-Length robuste), chacun couvert par un test (harnais Edge pour le XSS, tests/test_serve_wiki.py pour les 2 autres).

EXEMPLE CONSTATÉ	LOCALISATION
CORRIGE : XSS stocke (j.libelle/j.tail injectes en innerHTML sans echapper) — desormais passes par echapper() ; verifie par harnais Edge (charge onerror/script rendue comme texte, aucune execution ; FAIL confirme sur le code d'avant)	<code>scripts/scan_projets.py:carteRapport</code>
CORRIGE : Access-Control-Allow-Origin '*' -> restreint a Origin null (file://) + localhost/127.0.0.1 uniquement — une page web tierce ne peut plus POSTer vers ce serveur	<code>scripts/serve_wiki.py:_send</code>
CORRIGE : Content-Length malforme -> 400 propre (etait ValueError->500) + corps POST borne a 64 Kio ; 2 tests dedies dans tests/test_serve_wiki.py	<code>scripts/serve_wiki.py:do_POST</code>